

Projeto: Analisador de Logs e Central de Monitoramento de TI

1. Visão Geral

O objetivo deste projeto é construir uma ferramenta visual de auditoria e monitoramento de infraestrutura. A aplicação deve carregar e processar arquivos de log gerados por servidores ou sistemas de TI, apresentar métricas detalhadas em um painel (*dashboard*), emitir alertas com base em regras configuráveis e permitir o agendamento de tarefas/rotinas operacionais automáticas.

2. Objetivos de Aprendizado e Aplicação (POO)

Este projeto exercita conceitos fundamentais de desenvolvimento OO e manipulação de arquivos:

- **Entrada e Saída (I/O) e Manipulação de Arquivos:** Utilização das classes e fluxos de leitura (`FileReader`, `BufferedReader` ou APIs de `nio.file` como `Path` e `Files`) para carregar e interpretar arquivos de texto em disco.
- **Tratamento de Exceções e Resiliência:** Estratégias de `try-catch` e `try-with-resources` para garantir a leitura segura de arquivos, evitando corrupção de memória ou parada inesperada em caso de falhas de E/S (`IOException`, `FileNotFoundException`).
- **Encapsulamento e Modelo de Dados:** Estruturar classes bem delimitadas (como `RegistroLog`, `Métrica`, `Alerta`) protegendo os atributos via modificadores de acesso (`private`).
- **Padrão Observer / Modificadores de Eventos:** Aplicação do padrão *Observer* para escutar e reagir à detecção de eventos nos logs.

3. Funcionalidades Obrigatórias

A. Carregamento de Arquivos de Log

- Seleção de arquivos localmente usando componentes de caixa de diálogo gráfica (`JFileChooser` ou caixa de seleção JavaFX).
- Leitura sequencial e extração estruturada de dados (Data/Hora, Tipo do Evento: `INFO`, `WARNING`, `ERROR`, Endereço IP e Mensagem).

B. Dashboard de Monitoramento (GUI)

- Painel visual construído com biblioteca gráfica (Swing ou JavaFX) baseado no modelo MVC (Model-View-Controller).
- Exibição de estatísticas consolidadas: total de registros lidos, contagem por nível de severidade e gráfico/lista dos IPs mais recorrentes.
- Filtros interativos para busca dinâmica por palavras-chave ou intervalo de datas.

C. Configuração de Ações de Notificação e Rotinas

- **Ações de Notificação (Alertas):** Regras dinâmicas definidas pelo usuário (ex.: "Emitir alerta se o número de erros do tipo **ERROR** ultrapassar 5 em um minuto" ou "Detectar IPs suspeitos com falha de login").
- **Configuração de Rotinas (Agendamento):** Uso de temporizadores (**Timer** e **TimerTask**) para agendar varreduras periódicas em diretórios, atualizações do *dashboard* ou rotinas automatizadas de limpeza/backup de logs.

4. Responsabilidades (1 Alunos)

- **Interface Visual & Dashboard (View / Controller)**
 - Criação das telas e componentes da interface gráfica.
 - Implementação da integração com **JFileChooser** ou diálogo equivalente para carregamento de arquivos.
 - Atualização dinâmica dos elementos visuais e tabelas do *dashboard*.
- **Mecanismo de Leitura, Alertas e Rotinas (Model & Core)**
 - Lógica de parsing/extração de dados dos arquivos de log usando fluxos de leitura e tratamento de erros.
 - Estruturação das regras de notificação (Padrão Observer/Listeners).
 - Implementação das rotinas temporizadas via **TimerTask** e **Timer**.

Bônus: Integração com Inteligência Artificial (Análise Preditiva e Sumarização)

É totalmente viável integrar este sistema a uma IA! A inclusão de uma camada de IA eleva a complexidade do projeto sem exigir alterações na estrutura principal.

Como implementar:

1. **Sumarização Automática de Incidentes:** Enviar blocos de linhas de log que continham erros (**ERROR**/**CRITICAL**) via requisição HTTP (API REST de um provedor de LLM) para que a IA gere um resumo executivo em linguagem natural explicando o que provavelmente causou a falha no sistema.
2. **Análise Preditiva de Anomalias:** Usar uma API de IA/Classificação para analisar o comportamento do log e alertar o administrador sobre um padrão atípico de acesso antes mesmo que uma regra estática de notificação seja disparada.
3. **Sugestão de Solução de TI:** Integrar um assistente que lê a mensagem de erro do log de TI e sugere comandos ou etapas de solução de problemas diretamente na tela do usuário.

5. Entregas Esperadas do Projeto

Para concluir o projeto, os alunos deverão entregar e apresentar os seguintes artefatos:

- **Código-Fonte Completo e Compilável:** Organizado seguindo convenções de pacotes (separando as camadas de visão, controle, modelos e manipulação de arquivos) e sem erros de compilação.
- **Documentação Técnica do Projeto (README):** Contendo uma descrição das classes construídas, diagramas de arquitetura/classes, documentação externa e interna explicando as decisões de projeto, instruções claras de execução e um relatório sobre a utilização (se houver) de ferramentas de IA no desenvolvimento.
- **Massa de Teste e Logs de Exemplo:** Um conjunto de arquivos de texto formatados com registros de log (**INFO**, **WARNING**, **ERROR**) utilizados para testar as regras de notificação e demonstrar o funcionamento do *dashboard*.
- **Demonstração Prática do Sistema:** Apresentação da ferramenta em execução, comprovando o carregamento dos arquivos, a atualização em tempo real dos dados na interface gráfica, o disparo dos alertas configurados e o funcionamento do agendador de rotinas.